

Linguaggi di Programmazione

G. Michele Pinna

Univ. Cagliari

Anno Accademico 2011/12

Sintassi

La sintassi di un linguaggio di programmazione (abbastanza astratta) è la seguente

```
d ::= Intvar(i,n) | Boolvar(i,b) | Intfun(i,p,e) |  
      Boolfun(i,p,e)
```

```
p ::= Intpar(i) | Boolpar(i) | None
```

```
e ::= Eint(n) | Ebool(b) | Val(i) | Sum(e1,e2) |  
      Times(e1,e2) | Diff(e1,e2) | Minus(e) |  
      And(e1,e2) | Or(e1,e2) | Not(e) | Less(e1,e2) |  
      Equal(e1,e2) | Ifthenelse(e1,e2,e3) |  
      Let(d,e) | Apply(i,e)
```

n è un numero intero, b è un booleano ed i è un *nome*

Un'espressione da valutare è un elemento di e

Oggetti denotabili e ambiente

Sono gli interi, i booleani e, ovviamente, anche il codice delle funzioni (l'espressione da valutare) ed eventualmente l'ambiente non locale dove valutare l'espressione

l'ambiente associa ai nomi gli oggetti denotabili, e per convenienza è una coppia (δ_l, δ_{nl}) dove δ_l sono le associazioni *locali* e δ_{nl} quelle *non locali*

gli ambienti sono su una pila: quando introduco un nuovo ambiente faccio una `push` sulla pila del nuovo ambiente, mentre quando termino l'esecuzione di un blocco faccio una `pop` della pila. L'ambiente in cima alla pila lo trovo con `top`

quando incontro una dichiarazione, aggiungo all'ambiente locale le dichiarazioni (`insert(i,o,delta)` e `o` è l'oggetto denotabile), controllando nell'ambiente non locale se il nome `c`'è (`ispresent(i,delta)`) e nel caso ci sia cancellandolo dall'ambiente non locale (`delete(i,delta)`) e l'ambiente ereditato (sia locale che non locale) diventa il nuovo ambiente non locale (l'unione di due ambienti è la `union(delta1,delta2)`)

ad un elemento dell'ambiente accedo con `access(i,delta)`. Se l'elemento non è presente né in quello locale né in quello non locale, allora fallisce

cosa dovete fare

Date la semantica di tutto (dichiarazioni ed espressioni).

Dobbiamo dare la semantica alle dichiarazioni.

Una dichiarazione deve costruire l'ambiente a partire da un ambiente dato!

l'ambiente vuoto lo indico con $\emptyset$

$$\text{isPresent}(i, \delta_{\ell n}) = \text{false} \quad \delta'_\ell = \text{insert}(i, n, \delta_\ell)$$

$$\langle \text{IntVar}(i, n), \{\delta_e, \delta_{ne}\} \rangle \Rightarrow (\delta'_e, \delta'_{ne})$$

$$\frac{\text{isPresent}(i, \delta_{ne}) = \text{true} \quad \delta'_{ne} = \text{delete}(i, \delta_{ne}) \quad \delta'_e = \text{insert}(i, n, \delta_e)}{\langle \text{IntVar}(i, n), (\delta_e, \delta_{ne}) \rangle \Rightarrow (\delta'_e, \delta'_{ne})}$$

$\text{BoolVar}(i, n)$ segue lo stesso schema

Dichiarazione di funzioni! due casi: scope statico o scope dinamico.

Statico

Ho 4 regole (i, n assenti, uno dei due assente, nessuno).

$$\frac{\overbrace{\text{isPresent}(i, \delta_{ne}) = \text{false}}^{A_1} \quad \overbrace{\text{isPresent}(n, \delta_{ne}) = \text{false}}^{A_2} \quad \overbrace{\text{insert}(n, -, \delta_e) = \hat{\delta}_e}^{A_3}}{\langle \text{Intfun}(i, n, e), (\delta_e, \delta_{ne}) \rangle \Rightarrow (\text{insert}(i, (e, (\hat{\delta}_e, \delta_{ne}))), \delta_{ne})}$$

(regola tutte e due assenti)

ho dato un nome alle premesse (A_i) per abbreviare nelle altre regole...

$$\overbrace{\text{insert}(i, \delta_{ne}) = \text{true}}^{A_4} \quad A_2 \quad \overbrace{\delta'_{ne} = \text{delete}(i, \delta_{ne})}^{A_5} \quad A_3$$

$$\langle \text{Intfun}(i, r, e), (\delta_e, \delta_{ne}) \rangle \Rightarrow (\text{insert}(i, r, (\delta'_e, \delta_{ne})), \delta'_{ne})$$

(solo r assente)

C_1

$$A_1 \quad \overbrace{\text{insert}(r, \delta_{ne}) = \text{true}}^{A_6} \quad \delta'_{ne} = \text{delete}(i, \delta_{ne}) \quad A_3$$

C_1

(solo i assente)

$$A_4 \quad A_5 \quad A_6 \quad \delta''_{ne} = \text{delete}(i, \delta'_{ne}) \quad A_3$$

$$\langle \text{Intfun}(i, r, e), (\delta_e, \delta_{ne}) \rangle \Rightarrow (\text{insert}(i, r, (\delta'_e, \delta_{ne})), \delta''_{ne})$$

tutti e due presenti.

per Boolfun ho 4 regole analoghe.

Cosa succede ora per il $\text{let}(d, e)$ e $\text{Apply}(i, e)$?
Dato che ho una pila di ambienti, ora avrò

$$\langle e, \text{pila} \rangle \rightarrow v \quad (v \text{ è il valore})$$

(notate la freccia diversa!)

indichiamo la pila con σ

$$\langle d, (\emptyset, \text{union}(\text{top}(\sigma))) \rangle \Rightarrow (\delta_e, \delta_{ne}) \quad \langle e, \text{push}((\delta_e, \delta_{ne}), \sigma) \rangle \rightarrow n$$

$$\langle \text{let}(d, e), \sigma \rangle \rightarrow n$$

Provate a scrivere le altre....